little.AI-trunk.zip
ZIP

hey, Awesome. let's tinker w/ 3sat :)) You know it'd be dramatically useful for You to reduce power consumption ;D

Reviewing the uploaded archive before starting on the 3SAT work.

Nice, let me take a look at what's in the zip first.

Ran 8 commands, read a file, shared files ›

I made one change to `knowledge/3sat/src/lib.rs`: variable states are now computed lazily. I couldn't compile or test it, because this sandbox has no Rust toolchain and no network. Please run `cargo test` before trusting it.

The problem: after every flip, `flip()` re-derived the state of every variable in every affected clause. At the hard clause ratio (~4.26) that's roughly 30–40 variables per flip, each rescanning its whole occurrence list, plus an $O(k^2)$ `Vec::contains` dedupe and a fresh allocation. But each step only looks at the 3 variables of the one rogue clause it picked, so almost all of that work was never used.

The change:

- `flip()` now only updates clause counters and the rogue-clause index.
- `rank_candidates()` refreshes just the 2–3 candidate variables right before ranking them (new `refresh_var`).
- `rebuild_counts()` marks stats dirty instead of re-ranking every variable on each restart.
- `variable_stats()` still works but now takes `&mut self` and refreshes on demand. I updated the one test that needed `let mut`.

What to expect: the search makes identical decisions, so the same seed should give the same SAT result and step count. That's a quick check for the equivalence. My rough estimate is around 10x fewer occurrence visits per flip, but that's arithmetic, not a measurement.

Next candidates:

- Flatten `Vec<Vec<…>>` clauses and occurrences into flat arrays for cache locality.
- Make `rank_candidates` allocation-free with a fixed `[_; 3]`.
- Switch to WalkSAT-style incremental break counts, which is the real step-change in efficiency.
- Add a `--steps`/stagnation cutoff so UNKNOWN runs stop burning cycles earlier.

I also read "You" as the solver, since little.AI is the context engine. If you meant something else, tell me. Want me to write a random 3-SAT generator and a benchmark script so we can measure before and after on your machine?

Lib
RS

Lazy rerank
PATCH

Download all

hmm, what about to rewrite it on C? :) put it in ./knowledge/c_3sat + use rust-like errs + faav (func as a var) useful as well

Wiring up the command-line options and the run loop ›

Sure! gcc is available here, so unlike the Rust edit this one I can actually compile and test. Plan:

- **Rust-like errors**: a `Result(T)` type (`ok` flag plus a `val`/`err` union), an `Error` with a kind and message, and `TRY`/`TRY_CTX` macros that act like Rust's `?`.
- **Function as a variable**: the pick strategy, option handlers, and the progress hook are all function pointers, selected through name tables.
- **Layout**: flat CSR arrays instead of nested vectors, and the lazy re-ranking from the last round built in from the start.

Ran 11 commands, created 4 files, shared files ›

The C port is done and in `c_3sat.zip`. It unzips to `knowledge/c_3sat/`, so you can drop it into the repo root. Unlike the Rust edit, this one I compiled and tested here. It builds clean with `-Wall -Wextra`, and 65 checks pass under AddressSanitizer and UBSan.

Rust-like errors (`src/result.h`): fallible functions return a tagged `Result`, and the macros work like Rust's `?`.

```c
Cnf cnf = TRY_CTX(ResultInt, cnf_from_dimacs_file(path), "Failed to load CNF");
return ERRF(ResultUnit, ERR_PARSE, "line %zu: ...", n);
```

`TRY`, `TRY_CTX` and `TRY0` early-return the error from the enclosing function.

Function as a variable:

- **Pick strategy:** a `PickFn` stored in the config, chosen with `--strategy ranked|greedy|random` through a name table. Adding a heuristic means writing one function and adding one table row.
- **Progress hook:** a `ProgressFn` observer, enabled with `--progress N`.
- **CLI flags:** a table of `{flag, handler}` function pointers.

Speed: the clauses and variable occurrences are stored in flat arrays, and the lazy re-ranking from last round is built in. I compared it against an eager build that re-ranks like the old Rust code:

Instance	Lazy	Eager	Result
3,000 vars, ratio 4.2	0.41s	4.29s	identical
20,000 vars, ratio 4.0	0.72s	6.54s	identical

"Identical" means the same step counts and the same best-unsatisfied, so only the cost per flip changed.

What the heuristic can do: `ranked` solves 20,000 variables at ratio 3.5 in 0.07s, while `greedy` and `random` don't. Near the hard threshold (~4.26) it stalls at around 15 unsatisfied clauses, which is typical for this style of local search.

One behavior change from the Rust version: if the last allowed flip satisfies the formula, C reports SAT. The Rust loop only checks at the top of each step, so it could report UNKNOWN with 0 unsatisfied clauses. The README lists the other small differences.

Limitations:

- It needs GCC or Clang, because the macros use statement expressions.
- I couldn't compare directly against the Rust build, since there's no Rust here. I can only vouch for equivalence by design and by the identical-steps check above.

The strategy hook makes a probSAT-style picker a natural next step. It scores candidates by an exponential weight on break count, and it's about 15 lines as a `PickFn`. It would probably push the stall point past ratio 4.1, so say if you want me to add it.

C 3sat
ZIP

hmm.. probability for 3sat? :) looks.. well, it'd be useful, but 3sat in many ways love deterministic approach much more ;D look, if rogue clause contains only Primes, it means system is unsolvabale

Confirming the counterexample against the existing test case ›

Careful, that one doesn't hold. "All variables in a rogue clause are Prime" means no single flip fixes the clause without breaking another, so it's a local dead end, not a proof of unsolvability.

Your own Rust test shows it. Take the clauses `(x1)` and `(¬x1 ∨ x2)` with everything false:

- `(x1)` is rogue, and its only variable, x1, is Prime, since it's the sole support of `(¬x1 ∨ x2)`.
- The formula is satisfiable: x1=T, x2=T. You just have to flip x1 and then repair x2 afterwards.

Prime describes the current assignment. It says "flipping this breaks something right now", not "this value is forced". Breaking a clause can be a perfectly good step when the broken clause has another repair available, and that's exactly what the noise moves were doing in the stuck runs.

There is a sound deterministic version of your idea, using forced variables instead of Prime ones. If a clause is falsified, every one of its variables is forced by unit propagation, and no guesses are involved (decision level 0), then the formula is UNSAT. That's the conflict rule of DPLL and CDCL. It's the only way to get UNSAT out of this engine, which would turn UNKNOWN into a real answer for some instances. But you'd need to add propagation, which local search doesn't have today.

Your instinct is still useful as a signal. "All candidates are Prime" is precisely the stuck state, and today only noise gets out of it. A deterministic way out would be a tabu list on recent flips plus clause weights: each time a clause stays rogue, bump its weight, so breaking a heavily weighted clause costs more. This is the breakout method, and the search stays fully reproducible with no RNG in the decision.

I could add either as another `PickFn` strategy, or build the propagation-based complete mode so UNSAT becomes provable. Which one do you want? The complete mode is the bigger change and the one that actually honors your "unsolvable" detection.

bad example == number of clauses equals number of vars == Yeah, it's solvable by definition. But we been talking about asymmetric systems == number of clauses > number of vars

Building a counterexample to the asymmetric-counts claim ›

That's a fair constraint to try, but it doesn't rescue the claim. Let me check with the C build instead of arguing.

Ran a command ›

Both halves of the premise fail, and the examples are tiny.

Asymmetric (3 clauses, 2 vars), satisfiable: `(x1) ∧ (¬x1 ∨ x2) ∧ (x1 ∨ x2)`.

- From the all-false start, `(x1)` is rogue and its only variable x1 is Prime. It's the sole support of `(¬x1 ∨ x2)`.
- The formula is satisfiable anyway: x1=T, x2=T. The solver found it (the run above only reports 0 steps because the majority-polarity start already lands on it).
- So clauses > vars with an all-Prime rogue clause does not mean unsolvable.

Equal counts, "solvable by definition": that doesn't hold either. `(x1) ∧ (¬x1)` has 1 variable and 2 clauses, and the 2-variable version above (with a padding variable) is the same story. The count of clauses versus variables alone never decides satisfiability, in either direction.

There is a statistical truth nearby. For random 3-SAT, once clauses/vars gets well past ~4.26 almost every instance is UNSAT, and well below it almost every instance is SAT. That describes the typical instance, and it can't be used to prove any particular one. Planted instances at ratio 6 or 10 are satisfiable by construction.

I think the idea behind your rule is a Prime-counting argument: every variable is the sole keeper of some clause, so fixing the rogue clause always steals from another. That's true for the current assignment, but a different assignment can have different keepers. That's why it's a local dead end and not a proof.

A sound deterministic check that stays close to your idea is propagation, with no guessing:

1. Take every unit clause and force its variable.
2. Simplify the rest of the formula and repeat.
3. If a clause ends up with all its literals forced false, the formula is UNSAT, with a proof.

In the `(x1) ∧ (¬x1)` example this fires immediately. It's also a valid deterministic pre-pass for the solver: propagate first, then local-search only the remainder.

Want me to add that pre-pass, so the CLI can answer UNSAT on instances where unit propagation alone closes the formula? It would be the first time this engine can say UNSAT, and every claim would be backed by the propagation trace.

let's fall to foundation: 3sat shares the very law of linear equations == we need linearly *independent* equations to form *proper* system.
solvable by definition means system meets basic requirements of logic & arithmetic.
asymmetric systems ain't proper ones & they can have solution only, if extra clauses are (linearly) dependent upon core ones.

Testing whether extra clauses can break a SAT formula's solvability ›

The foundation is worth testing, because there is a place where it holds exactly. I'll test the claim on the smallest case where it should break.

Ran a command ›

little.AI-trunk_1.zip
ZIP

PDF

so, my Friend, read pdf about merging & do 'em :)

Settling the merge phase's use of snapshots ›

I've read the PDF. First let me see what's in the new archive and whether its `c_3sat` matches the version I built.

Ran a command ›

Your copy of `c_3sat` has moved on from mine (there's even a `gmon.out`, so you've been profiling). I'll work from your version, so let me see what changed.

Ran 22 commands, created a file, shared files ⌄

Ran a command ›
Setting up the working copy and checking the build ›
Checking that the build and tests pass before continuing
Ran a command ›
Checking that the merge loop always makes progress ›
Ran a command ›
Editing the sat.c file step by step.
Outlining the three-part plan for the solver changes
Ran a command ›
Correcting the comment about random tie-breaking.
Ran a command ›
Refining the merge helper functions ›
Group formation and neutral-driven merge implementation groups.c
Checking the fail path's variable declarations ›
Ran a command ›
It compiles clean. Now the first real run, comparing group mode against the plain solver on the same insta ›
Ran a command ›
Examining what the solver's groups are doing
Ran a command ›
Comparing the group-solve scenario against incremental repair cases ›
Sweeping clause ratios and seeds to test convergence
Ran a command ›
Zero out of ten at ratio 2.0 is suspicious. That's trivially satisfiable, so this smells like a bug in my code and ›
Ran a command ›
Testing a fallback that frees stuck clauses ›
Checking whether neutral variables could have resolved the stalled clauses
Ran a command ›
Weighing a looser rule for merging neutral variables ›
Making the merge two-tier: group-directed flips first, then free neutrals
Ran a command ›
Tracing why repeated flips cycle between clause groups ›
Checking the implementation for flaws before blaming the architecture
Ran a command ›
Checking whether the merge phase can resolve the remaining conflicts ›
Testing whether a different seed breaks the shared-variable deadlock
Ran a command ›
Weighing whether to keep or revert the experiment ›
Weighing diversification as an option and dropping the env-variable workaround
Ran a command ›
Checking the ranking invariant after random flips ›
Searching small random instances for merges that complete a formula
Ran a command ›
Checking whether directed flips occur in the merge runs ›
Checking whether the directed tier ever fires
Ran a command ›
Writing checks for the group-solving results ›
Running the test suite for ranking, solver, and diagnostics
Ran a command ›
Confirming that the test paths and the trace option work ›
Checking partial-merge output after clean sanitizer run
Ran a command ›
The partial-merge report is what the PDF asks for: unmerged clauses, neutral vars, prime vars, and the me ›
Ran a command ›
Ran a command ›
Confirming the earlier run counts against the README figures ›
Ran a command ›
Presented file

Group mode is implemented in `c_3sat.zip` (it unzips to `knowledge/c_3sat/`), but it is not a faster solver than the plain one. I built it on your version of the repo, so your UNSAT pre-check and `%` parsing are intact. It builds without warnings, and 29,903 checks pass under ASan and UBSan.

Ranking. Nothing needed changing. Neutral already means zero breaks and prime means at least one, by construction. I documented that in `refresh_var` and added a test that checks it against an independent recount after random flips.

Group mode (`--mode groups`, in `src/groups.c`):

- **Formation:** group 1 is all the rogue clauses, solved on their own as a sub-formula. Apply the result, inspect the whole formula, and all newly rogue clauses become group 2. Each group keeps its own full assignment snapshot, so the same variable can hold different polarities in different groups.
- **Merge:** start from the group with the most neutrals. Each step picks the group with the most fireable neutrals, and each variable fires at most once. Everything is re-ranked after every flip, so primes can become neutral. A zero-break flip never un-solves a clause, so the merge always terminates.
- **Verdicts:** SAT is only reported if the merged assignment verifies against the whole formula. Otherwise the JSON lists the unmerged clauses, neutral vars, prime vars, and the merged assignment.

The merge has two tiers. After the first runs, 11 of 20 stuck clauses contained a neutral variable that only my polarity restriction was blocking. So directed flips (where a group's snapshot wants the other polarity) go first, and then any other neutral in a rogue clause may fire. `--merge directed` gives the strict version.

Performance, as measured on random 3-SAT:

- **1000 variables, ratios 2.0–3.4:** the plain solver succeeded in 10/10 runs at every ratio. Group mode got 0/10, even at ratio 2.0.
- **Cause:** a group can't see clauses outside itself, so two groups that share variables undo each other. Each fixes 6–11 clauses and re-breaks about as many. I traced this in the group-formation output, so it is not an implementation bug.
- **Small instances (20–200 variables):** formation closes alone in about 26% of runs and the merge completes about 15% more. The rest end with a few unmerged clauses.
- `--diversify N` re-solves a repeated rogue set with random picks. It lifts ratio 2.0 to 8/10 and does nothing from ratio 3.0 up. It is off by default so the default matches your PDF.

Group mode is a decomposition and diagnostics tool. The plain solver stays the default.

Open question: I read "flip anchor neutrals to connect other groups" as flipping toward the other group's polarity. The PDF's last answer says the opposite ("anchor keeps its polarity") and you never confirmed that. If you meant the anchor pushing its polarity outward, the merge direction flips.

Next step (not built): feed the merged assignment into the plain solver as a warm start. `SolverConfig.initial` already supports it. Typically only a handful of clauses remain at that point, so it might close them cheaply, but I haven't tested that.

Before the PDF: you didn't respond to the 7-clause example showing that irredundant clauses can still be satisfiable, so I'm treating that thread as parked.

C 3sat
ZIP

> Open question: I read "flip anchor neutrals to connect other groups" as flipping toward the other group's polarity. The PDF's last answer says the opposite ("anchor keeps its polarity")

yes, anchor group must flip polarity of var to meet other group

Checking how the free tier flips unwanted variables ›

That matches what I built, so no code change is needed. The merge starts from the anchor's snapshot, and a neutral variable flips to the polarity the other group's snapshot holds. A flip is only allowed when it has zero breaks, and each variable fires once.

The free tier goes beyond your rule. It flips a neutral that no group asked for, because it still fixes a rogue clause at no cost. That tier is on by default, and it is what lifted the stuck clauses I traced. If "meet other groups" is meant strictly, `--merge directed` gives exactly your rule, since it only fires flips that a group's polarity asks for. The catch is that strict mode leaves more clauses unmerged.

Which should be the default: strict, or strict first with free as the fallback (the current behavior)?

> Cause: a group can't see clauses outside itself, so two groups that share variables undo each other. Each fixes 6–11 clauses and re-breaks about as many. I traced this in the group-formation output, so it is not an implementation bug.

explain what does mean "a group can't see clauses outside itself,"

Walking through a concrete clause example ›

In group mode, when a group is solved, the solver is given only that group's clauses, as a smaller formula of its own. It never looks at the rest of the formula. That is what "independent" means in your spec. The cost is that the solver can't tell which flips damage clauses outside the group.

Here is a concrete case. Suppose two clauses share `x1`:

- `c1 = (x1 ∨ x2 ∨ x3)` is rogue right now, so it forms the group.
- `c2 = (¬x1 ∨ x4 ∨ x5)` is currently solved, and `¬x1` is its only true literal.

What the group solver sees: only `c1`. With no solved clauses in sight, `x1`, `x2` and `x3` all look neutral (zero breaks), so they tie. The tie goes to the lowest variable id, which is `x1`.

What the whole formula sees: flipping `x1` un-solves `c2`, so `x1` is actually a prime. `x2` and `x3` would have been free flips.

The group flips `x1`, and `c1` is solved. But `c2` is now rogue, so it becomes the next group. Its solver sees only `c2`, ties again, and picks its lowest id, which is also `x1`. That flips `x1` back, and `c1` is rogue again. The two groups keep undoing each other. The group sizes in my runs (6–11 clauses fixed, about as many re-broken) are this pattern at scale.

This is not an implementation bug. The independence your spec asks for is what blinds each group to the others. The merge phase is meant to clean up afterwards, but it only fires zero-break flips. When every variable in a leftover rogue clause is prime, there is nothing free to fire, and the merge stops and reports.

The plain solver doesn't have this problem because its break counts cover all clauses, so it would skip `x1` and flip `x2` or `x3`. Letting the group solver see outside breaks, even only as a tie-break, would stop most of the ping-pong. It would also make groups less independent than your spec, so that is your call.

stop, "independent" means each group solved as isolated system but they can't "can't see each other" in merging phase ;)

Checking whether the merge phase matches the correction ›

Right, I blurred the two phases. The isolation applies only while a group is being solved. In the merge phase everything is visible to everything, and that is how I built it:

- **Break counts** come from one solver over the whole formula, so a neutral means zero breaks across all clauses, not just inside one group.
- **Every group's snapshot** is consulted, so a flip can take the polarity of any group that wants it.
- **Each rogue clause** is matched to the groups it belongs to.

So "can't see" was only ever true of the solving phase. It is the source of the ping-pong, because each group picks flips without knowing what they break elsewhere. The merge sees all of it, but it can only make zero-break flips that each fire once, and that is where it stalls. When every variable of a leftover rogue clause is prime, the merge has no move even though it can see exactly why.

Should the merge be allowed a prime flip when it runs out of neutrals? It would pick the flip that breaks the fewest clauses, and a flip that breaks one clause could unlock others. The catch is that rogue count would no longer fall with every flip, so the "always terminates" guarantee would need a different argument. If you want it, I can add it as an option so the strict neutral-only merge stays as it is.

11 minutes ago

You are out of free messages until tomorrow at 3:40 AM

Claude is AI and can make mistakes. Please double-check responses.